

Rachis9 — A Tutorial

Rachis9 — A Tutorial

A working tour of R9, the language for autonomous real-world systems, as it stands on 2026-09-18. It follows `autonomous_real_world_language_design.md` and cites its sections, so anything here can be chased back to the reasoning behind it.

How to read this

Nothing here runs yet. There is no R9 compiler. The point of writing programs against this tutorial is to find out where the language is awkward, ambiguous, or missing — which is what turns a design into a language.

Everything is marked, because not all of it is equally firm:

mark	meaning
(settled)	an architectural decision; changing it would change the language
(provisional)	the shape is agreed, the spelling may change
(open)	named but undesigned; write around it

Chapter 13 lists what is not defined yet. Read it before writing your first program, so you know which walls are real.

1. The shape of a program

An R9 program has three layers. **(settled, §2)**

PROACTION	decide what should happen next
	requests states; observes results
REACTION	reach known states, deterministically
	writes inputs; plans over a fixed graph
REALTIME	do the physical thing, on time
	owns devices; publishes reality
RV-9	admits, schedules, enforces, measures

Authority runs *downward*: REALTIME constraints outrank REACTION, which outranks PROACTION **(settled, §31.2)**. Intelligence proposes; the deterministic machinery keeps control of what can safely happen.

The split is not “planning versus execution” — both upper layers plan. It is what can be proven and bounded versus what cannot, and the one-line version is:

REACTION knows only what reality shows. *PROACTION* knows what it has been through. (§31.1)

A program compiles to a *bundle*: each REALTIME component becomes an independently admitted RV-9 program, with a manifest saying what it needs. RV-9 refuses anything it cannot promise. (§2.1, §16.1)

2. Values have meaning and representation

Two different things, kept separate. (settled, §5)

```
let distance = 25m           # meaning: length
let delay    = 10ms
let voltage  = 12V
let throttle = 40%
let speed    = 10m/s
let g        = 9.81m/s2
```

Machine representation is written after a colon, and only when it matters:

```
let altitude = 12000m:f64
let ticks    = 40%:u8
```

The compiler does dimensional algebra, so relationships work without the language knowing any physics by name:

```
let current = 12V / 60hm      # current
let travel  = 10m/s * 2s      # length
```

Mixing dimensions is an error. Adding a current to a voltage is not a runtime surprise; it does not compile.

Units are SI only. Structure is separate again — speed and velocity share a dimension, but one is a scalar and the other a vector:

```
let position = [10m, 5m, 2m]
let velocity = (position - previous) / dt
```

Text, instants and angles

(settled, §4.1, §5.1)

```
let label = "left motor"
```

Text is UTF-8 bytes with a length, and its **capacity is part of its type**: `text[16] + text[8]` is a `text[24]`, so nothing allocates and the compiler checks it fits. Escapes are `\n`, `\t`, `\\`, `\"` and `\u{...}`, and an unrecognised escape is a compile error rather than a surprise. Offsets are byte offsets, and there is no `char` type.

Two distinctions that dimensions alone do not make:

- an **instant** is a point in time, while a duration is a quantity. `instant - instant` gives a duration; `instant + instant` is an error;
- an **angle** has a dimension of its own, rather than being dimensionless as SI has it. Otherwise torque and energy are indistinguishable, and an angle can be added to a bare number unchallenged.

Arrays are storage, not matrices

```
f32[3,3]           u8[480,640,3]           f32[1024,3]
```

Arrays have any number of dimensions, and below PROACTION their sizes are compile-time constants. They deliberately do **not** define `*`, because elementwise and matrix-product are both defensible readings and either one silently betrays half its users. `matrix<3,3>`, `vector<3>`, `quaternion` and `transform` are library types over array storage, once parametric types exist (§4.2, §44.3).

Try this: write five declarations whose dimensions the compiler must infer, and one that should fail. Decide what error you would want to read.

3. Ordinary code

Familiar, deliberately. (§3, §8, §9)

```
let x = 5                # mutable
const MAX_SPEED = 2m/s   # immutable

if temperature > 80C
  ...
else if temperature > 60C
  ...
else
  ...
end

while pressure < target
  ...
end

for i = 0 to 9
  ...
end

loop                    # deliberately forever
  ...
end
```

`loop` exists so that an endless loop is always visible as a choice, never an accident (§9).

Waiting is a first-class statement, and a bounded wait is the normal kind:

```

await imu.ready within 100us
else
  fault IMU_TIMEOUT
end

```

`await` blocks until the condition holds. If `within` expires first, the `else` branch runs. (§10)

Functions and modules

(settled in shape; syntax provisional, §44.1)

```

module geometry

  export function distance(a: position, b: position) returns length
    return magnitude(b - a)
  end

  function magnitude(v: position) returns length      # private
    ...
  end

end

use geometry          # geometry.distance(a, b)
use geometry.distance # distance(a, b)

```

`returns` rather than `->`, because the arrow already means a transition between states. Declarations are private until `exported`, and an exported function must declare its result type while a private one may leave it inferred.

Functions are ordinary declarations, grouped in **modules** that serve as namespaces. Related things stay together because they are declared together — not because they hang off an object.

R9 is deliberately not method-oriented. It replaced *call the object* with *write an input, read a publication* (chapter 5), so `MOTOR.stop()` is not how anything works. But a call may be written either way:

```

distance(a, b)      # these two mean
a.distance(b)       # exactly the same thing

```

so chaining reads well — `v.normalize().scale(2m)` — while symmetric operations such as `distance` and `dot` stay symmetric. Operators carry the rest: `transform * point`, and all quantity arithmetic.

Function values exist only in PROACTION (§44.2). Below it, a choice of implementation — this IMU driver or that one, a real motor or a simulated one — is made when the bundle is built, so calls stay direct and the compiler’s guarantees (chapter 12) stay exact.

There is no inheritance and no dynamic dispatch.

Try this: rewrite a polling loop you have written in C as an `await` with a bound. Notice what you had to decide that C let you leave vague.

4. REALTIME: a component

A component is a unit of periodic physical work. **(settled, §11-§13)**

```
realtime MOTOR every 1ms
  deadline 800us

  input target_speed = 0m/s      # written by REACTION (provisional default)

  let speed = encoder.speed
  let error = target_speed - speed
  let output = pid(error)

  limit output to -75% .. 75%

  motor.power = output

  expose speed, output
end

failsafe MOTOR
  motor.power = 0%
end
```

Line by line:

- `every 1ms` is the release period. A component may instead be released by a hardware signal, and then must declare how fast that signal may arrive:

```
realtime LIMIT_SWITCH on gpio.rising
  minimum_interval 200us
  ...
end
```

Every component must declare one or the other — otherwise “late” has no meaning, and RV-9 cannot admit it **(settled, §15.6)**.

- `deadline 800us` is when the work must be finished. Omitted, it is the period.
- `input` is the only way anything outside can affect the component **(§19)**. Private state is private.
- `limit` clamps a value to a range, and applies to whatever REACTION asks for. A layer above cannot talk the component out of its own limits.
- `expose` publishes a coherent set (chapter 5).
- `failsafe` is not code that runs in the component. It is a *declaration* of safe values that RV-9 applies **after** the component is gone — the point being that a wedged control loop cannot be trusted to clean up after itself **(settled, §15)**.

There is no `priority`. RV-9 derives scheduling order from the whole admitted workload, and a component that must sit above or below the radio says so as a constraint, not a number **(settled, §13.1)**:

```
placement urgent          # never below the radio (provisional spelling)
```

Orderly stop is different from failure:

```
on stop MOTOR
    motor.power = 0%
end
```

`on stop` runs while the component is still healthy. A *fault* never runs it (**settled, §15.4**).

Bounded execution

Inside an admitted component, the code must be analysable: bounded loops, no unbounded waits, and only operations RV-9 marks real-time safe. Setting up — opening devices, sizing buffers — happens *before* the timing contract begins (**§14, §16.1**).

Try this: write a component that samples a sensor at 5 ms, computes a filtered value, and exposes it. Then decide what its failsafe should be and why. Most of the design work is in that second question.

5. Publishing, and being observed

`expose` is not an access modifier. It is a **publication** — an atomic, coherent snapshot (**settled, §18**).

```
expose speed, current, temperature
```

Observers see all three change together, never a mixture of old and new. Before the first publication, a value is simply not available yet.

Underneath, each set is a fixed-size cell on RV-9, carrying a sequence number, an observation timestamp and the values. The writer never waits and never allocates; all the cost of contention falls on the reader. A cell outlives its publisher, so the last thing a dead component said is still readable — which is what fault handling needs (**§18.1**).

Outside the component, published values are reached by name:

```
MOTOR.speed
MOTOR.output
```

and inputs are written the same way, by REACTION only:

```
MOTOR.target_speed = 2m/s
```

6. When things fail

`fault` stops the smallest enclosing construct that has a declared outcome, makes the physical world defined, and publishes why — **in that order (settled, §15.1)**:

```
fault SENSOR_LOST
```

1. the component's declared `failsafe` is applied;
2. the fault is published in the component's own cell;
3. the component is not released again.

The ordering matters: an observer must never learn of a fault before the actuators are safe, because it may react by commanding something.

It is not an exception. There is no unwinding, no handler, no propagation, and R9 has no `try/catch` (§39).

Faults nobody wrote are the same mechanism (§15.3): a missed deadline or a loop that stops responding is `DEADLINE`, a blown stack is `STACK`, a device failure is `DEVICE`, physical overflow is `OVERFLOW`.

Recovering is not a keyword. A fault is published state, so it is watched like anything else (§15.5, §30):

```
watch MOTOR.faulted
  if MOTOR.faulted
    transition SAFE
  end
end
```

7. REACTION: watching

`watch` observes **references, not conditions** — which is what gives the compiler a dependency graph, so nothing polls (settled, §21):

```
watch MOTOR.temperature, BATTERY.voltage

  if MOTOR.temperature > 80C
    MOTOR.power_limit = 40%
  end

  if BATTERY.voltage < MIN_VOLTAGE
    transition SAFE
  end

end
```

A watch body is a **reflex**: short, bounded, and never a place to orchestrate work. It may not use `in parallel`, directly or through anything it calls (§21, §14.1).

Silence

A physical system must also notice when an expected publication *stops*. Since silence changes nothing, a plain watch never runs, so the timeout form exists (**provisional, §21.1**):

```
watch IMU.orientation within 50ms
  process_orientation()
else
  transition SAFE          # nothing has been published for 50ms
end
```

Try this: which of your sensors would you want a silence branch on, and what should happen? This question tends to expose whether a design has a safe state at all.

8. States are predicates

A state is **not** a stored label. It is a predicate over published reality, and it is true whenever reality satisfies it (**settled, §23-§24**):

```
const STILL = 0.01m/s

state STOPPED
  abs(DRIVE.speed) < STILL
end

state SAFE
  abs(DRIVE.speed) < STILL
  BRAKE.engaged
  DRIVE.powered == false
end
```

Because a state is re-observed rather than remembered, planning and recovery always start from what is actually true — never from a label that might be stale (**§31.4**).

Tolerance

Measured quantities are never exactly anything, so exact equality on a continuous physical value is rejected in predicates (**provisional, §23.1**). Write a bound, and name it:

```
abs(DRIVE.speed) < STILL          # not: DRIVE.speed == 0m/s
```

Parameterized states

One declaration can describe a family (**provisional, §23.2**):

```

const ARRIVED = 5mm

state AT(target)
  abs(DRIVE.position - target) < ARRIVED
  abs(DRIVE.speed) < STILL
  BRAKE.engaged
end

```

The graph stays finite: parameters are values bound by a request, not new states.

Try this: define three states for a machine you know, such that between them they cover every reality it can be in. Coverage is what keeps a plan from having nowhere to start.

9. Transitions

A transition says how one known transformation happens (**settled, §25**):

```

transition STOPPED -> SAFE

  BRAKE.engage = true
  await BRAKE.engaged within 500ms
  else
    fault BRAKE_TIMEOUT
  end

  DRIVE.enabled = false

end

```

Requirements

require decides whether a transition is **eligible at all**. A transition whose requirement is false is not an edge the planner can use (**settled, §25.1**):

```

transition SAFE -> AT(target)

  require BATTERY.charge > MIN_CHARGE
  require target >= TRACK_START and target <= TRACK_END

  ...

end

```

if branches at runtime inside a body already chosen; **require** decides whether it can be chosen. That difference is the whole point, and it is what lets a failure say *which* requirement blocked the way, instead of an opaque “no path”.

Doing things at once

Some transformations are inherently concurrent — a boxing combination, an arm raised while driving (**provisional, §25.2**):

```

in parallel
  in sequence
    ARM.target = BIN
    await ARM.at_target within 10s
  else
    fault ARM_TIMEOUT
  end
end

  in sequence
    CONVEYOR.speed = 0.2m/s
    await CONVEYOR.ready within 8s
  else
    fault CONVEYOR_TIMEOUT
  end
end
end

```

Branches **must not interfere**: no two write the same input, and none reads what another writes. The compiler checks it, so overlapping writes are a compile error rather than a race. Given that, every interleaving gives the same result — concurrency without losing determinism.

A fault in one branch stops the others at their next safe point. These are not threads: a branch runs to its next `await`, which is exactly where it is safe to leave it.

10. Requesting, planning, and who wins

A request names a desired state, not a path (**settled, §26-§27**):

```

transition SAFE
transition AT(3.2m)

```

The planner works out how to get there from whatever is true now, using the eligible transitions. It is the SQL move: say what you want, let the system find the way.

Results are ordinary observable values (**§29**):

```

let attempt = transition SAFE

watch attempt.status
  if attempt.status == FAILED
    ...
  end
end
end

```

reason	when
NO_PATH	no sequence of transitions leads there at all
PRECONDITION_FAILED	paths exist, but requirements block every one
TIMEOUT	a bound was exceeded
COMPONENT_FAILED	something it depends on faulted
CONSTRAINT_VIOLATION	reserved
PREEMPTED	superseded by higher authority

Preemption

REACTION outranks PROACTION, so a safety response supersedes a pursuit. The superseded attempt ends `PREEMPTED` — but only at a **safe preemption point**, where stopping cannot leave a half-issued command (**settled, §31.3-§31.4**). Transition boundaries and every `await` are safe by construction.

Order matters for exactly this reason:

```
DRIVE.enabled = true      # the drive takes hold first
BRAKE.engage = false     # and only then does the brake let go
```

Stop between those two lines and the cart is held by both. Reverse them and it is held by neither.

Afterwards, planning starts from **newly observed reality** — which is why states are predicates rather than labels.

A safety response that must keep holding is expressed with `require`, not with a lock: while charge is low, no edge out of `SAFE` is eligible, so a request fails `PRECONDITION_FAILED` naming the requirement. PROACTION cannot override it, and it learns exactly why (**§31.3**).

11. PROACTION: deciding what to do

PROACTION has **no grammar yet (§34)**. What follows is its architecture, which is settled, with shapes written as sketches. This is the layer to think hardest about while reading.

One serialized authority. Decisions and changes to authoritative state pass through a single point. The computation behind them — vision, route search, retrieval, consulting a remote model — may run concurrently, on other cores or other machines (**settled, §34.3**):

Serialize authority, not intelligence.

What computes does not commit. A late result may be discarded; a failed computation is a value, not a fault.

Pursuits may proceed at once where they need different **authority domains** — navigation, manipulation, vision, communication. Two pursuits needing the same domain

contend: first come, first served, and losing is an ordinary attempt result, never an error. An autonomous system is never stopped by a refusal; it is informed by one (§34.3).

Decision points are where the authority takes up what has arrived and may change course. A pursuit may not run arbitrarily far without reaching one, so redirection has a bound. The authority must never block.

A PROACTION watcher informs; it does not take control. Only REACTION watchers act directly.

Several machines are several PROACTIONS. REALTIME and REACTION stay machine-local, because a bounded planner reading local cells cannot survive a network. A remote peer is a PROACTION-level requester: it asks for states and observes publications, and never writes another machine’s inputs. So a partition cannot make a machine unsafe — only idle (§45).

Idleness is a condition to watch. A machine pursuing nothing is safe but not autonomous, so “nothing is being pursued” deserves a declared fallback.

What PROACTION remembers

Ordinary program data: **records** and **fixed-size arrays** (**settled in direction**, §34.4).

```
observation.temperature  
observations[3]
```

And one **log** — the history the machine reasons from (**settled**, §34.4):

- an entry is **bounded text with attributes**; `time` is the first attribute, assigned by the store;
- the log is a **FIFO with a declared limit in entries**, with a maximum entry length, so `entries × (max length + attributes + header)` is known before it runs and RV-9 can admit it;
- when it is full the oldest go, and **curtailment is recorded**, so a reader can tell “nothing happened then” from “that is no longer here”;
- **reading is a projection**: choose which attributes to show, and show times absolutely or relative to a chosen instant or entry:

```
-5.904s  Battery voltage dropped  
-3.212s  Motor current increased  
0.000s   Traction lost  
+0.287s  REACTION entered SAFE
```

- whatever a projection shows, text search can match, so metadata needs no second query language;
- an attribute may be **anchored** to a span of the text, so `wheel slip 0.42` reads as a sentence and is reachable as a number (**provisional**);
- every attribute carries its **origin**: *assigned* is the store’s knowledge, *declared* is a writer’s claim, and code cannot silently treat one as the other.

Try this: write out twenty log lines your machine would produce during a failure, then ask what you would want to search for at 3 a.m. That is the best test of this design that exists right now.

Learned behavior moving down

What PROACTION learns may change what it *requests*. It never edits the compiled transition graph at runtime. A stabilized behavior becomes candidate R9 source, and goes through analysis, compilation and deployment like anything a person wrote (**settled, §37**).

12. What the compiler does for you

- **dimensions** — physical meaning checked at every operation (§5);
- **bounds** — established, declared, or observed, and never presented as each other (§14);
- **non-interference** — parallel branches that would collide (§25.2);
- **properties through the call graph** — a rule about a construct is worthless if a call can smuggle the construct past it, so restrictions are propagated to whatever reaches them (§14.1);
- **the resource contract** — periods, deadlines, stacks, devices, publications and failsafes, emitted as a manifest RV-9 admits against actual hardware, then measures (§16.1, §2.1).

A library is coloured the same way: geometry is real-time safe, a planner is not, and a library carries that outward to whatever calls it (§44.3). That is what stops a library call from quietly breaking admission inside a control loop.

R9 describes; RV-9 enforces. Where they disagree, RV-9 reports it as ordinary observable state.

13. Not defined yet

Write around these; they are the walls that are still missing, not walls you have hit.

missing	what to do meanwhile
function and module syntax	provisional and usable (§44.1); the libraries themselves do not exist, so call <code>pid</code> , <code>abs</code> and <code>distance</code> as if they did
parametric types	wanted by the first libraries (<code>estimate<T></code> , <code>matrices</code>); undesigned (§44.3)
coordinate frames	dimensions are checked, frames are not yet; keep frames in your naming (§44.3)
record and type declarations	use them by name and note what you wanted
matrices, vectors, quaternions	library types awaiting parametric types; use arrays and note what you wanted (§4.2)
device binding	<code>encoder.speed</code> is assumed bound to an RV-9 path by the build (§42)
program structure	no files, modules, imports or namespaces yet
PROACTION grammar	write it as prose or pseudo-code beside the program
<code>input</code> , <code>placement</code> , failsafe spelling	provisional — use as shown

The most useful thing you can do while reading is to **note every place you wanted syntax that did not exist**, and what you would have written. That list is the next chapter of the design.

Appendix: the whole vocabulary

general	<code>let</code> <code>const</code> <code>if</code> <code>else</code> <code>for</code> <code>while</code> <code>loop</code> <code>await</code> <code>within</code> <code>return</code>
REALTIME	<code>realtime</code> <code>every</code> <code>on</code> <code>minimum_interval</code> <code>deadline</code> <code>placement</code>
	<code>input</code> <code>expose</code> <code>limit</code> <code>failsafe</code> <code>fault</code> <code>on</code> <code>stop</code>
REACTION	<code>watch</code> <code>state</code> <code>transition</code> <code>require</code> <code>in</code> <code>sequence</code> <code>in</code> <code>parallel</code>
PROACTION	<code>in</code> <code>sequence</code> <code>in</code> <code>parallel</code> (nothing else yet)

Deliberately absent: `event`, `heal`, `recover`, `proof`, `invariant`, `mutex`, `semaphore`, `thread`, `lock`, `malloc`, `free`, `try`, `catch`, `priority`.

Libraries are further off still, and Appendix A of the design document lists the ones R9 will eventually want, in the order they would be built. Tier 1 — math, geometry, quantities — is what forces the language questions above, so it is really a language decision wearing a library's clothes.

If something cannot be said with what is there, that is worth writing down — but the design's standing rule is to resist new vocabulary until a concrete problem cannot be expressed cleanly without it (§39).